

Ghostty Tmux Control Mode: Deep Understanding (Sections 1–5)

Purpose: Build foundational understanding of tmux, tmux control mode, and Ghostty's current state. This is research — not a code proposal.

Evidence classification used throughout:

- **(doc)** = from official documentation or man pages
- **(code)** = verified by reading Ghostty source code
- **(issue)** = from GitHub issues, PRs, or discussions
- **(observed)** = from runtime experiments
- **(inferred)** = reasonable conclusion not directly confirmed — labeled explicitly

Section 1: First-Principles Explanation of tmux and Control Mode

1.1 What is tmux?

tmux is a **terminal multiplexer**. It sits between your shell (e.g., zsh, bash) and your terminal emulator (e.g., Ghostty, iTerm2, Terminal.app).

What it does:

- **Multiple sessions in one terminal window.** You can have many shell sessions — tabs, split panes — all inside one terminal window, managed by tmux.
- **Session persistence.** If your terminal closes (laptop lid closes, SSH disconnects), the tmux server keeps running. You can reattach later and everything is exactly where you left it.
- **Remote session survival.** When working over SSH, tmux lets you detach from a session and reattach from a different machine or connection.

tmux is a **server-client architecture**:

- The **tmux server** runs in the background. It manages windows, panes, and the actual shell processes.
- A **tmux client** connects to the server. A client is what you interact with. When you type **tmux** in your terminal, you start a client.

(doc) Source: `tmux(1)` man page.

1.2 How "normal tmux in a terminal" works

When you run **tmux** in a terminal like Ghostty:

1. tmux starts a server (if one isn't running) and a client.
2. The client takes over your terminal window.
3. tmux draws its own UI inside your terminal: a status bar at the bottom, split panes, window indicators.
4. tmux reads your keystrokes and routes them — some go to the shell running inside a pane, some are intercepted by tmux itself (e.g., **Ctrl-b c** to create a new window).
5. tmux sends VT100/xterm escape sequences to your terminal emulator to draw pane content, borders, status bars, etc.

In this mode, tmux is just another program running inside the terminal. The terminal emulator (Ghostty) has **no idea** that tmux is running — it just sees a stream of escape sequences. Ghostty cannot:

- Know how many panes tmux has
- Provide native tabs or split views for tmux windows
- Let you use OS-level scrollback, search, or selection inside tmux panes
- Offer native window management (drag-to-resize, native keybindings)

The terminal is "dumb" with respect to tmux — it just renders whatever bytes tmux sends.

1.3 What is tmux control mode?

Control mode is an **alternative client interface** to the tmux server. Instead of tmux drawing a TUI inside your terminal, tmux sends **structured text messages** over stdout that describe what is happening in the session.

(doc) From the tmux wiki (<https://github.com/tmux/tmux/wiki/Control-Mode>):

Control mode is a special mode where tmux accepts commands as lines of text on standard input and outputs notifications as text on standard output.

Key differences from normal mode:

Aspect	Normal tmux	Control mode
UI drawing	tmux draws its own TUI in your terminal	tmux sends structured text messages
Pane borders	Drawn by tmux as box-drawing characters	Not drawn — the client decides how to display panes
Status bar	Drawn by tmux at the bottom	Not drawn — the client gets session/window info as data
Scrollback	Managed by tmux internally	Pane content sent as data to the client
Input routing	tmux intercepts keystrokes	The client decides what to send to tmux
Who's in charge	tmux controls the visual layout	The client application controls everything visible

Control mode turns tmux from "a TUI app that takes over your terminal" into "a session management protocol that a smart client can drive."

1.4 How to enter control mode

There are two flags **(doc)**:

- `tmux -C` — starts control mode with echo enabled (for human testing). You see what you type. Useful for debugging.
- `tmux -CC` — starts control mode with echo disabled and most terminal features off. This is what terminal emulators use.

You can also attach to an existing session:

```
tmux -CC attach -t mysession
tmux -CC new-session -s work
```

1.5 What you would see if you ran `tmux -CC`

If you open a shell and run `tmux -CC`, you would see something like:

```
%begin 1711000000 0 0
%end 1711000000 0 0
%session-changed $0 default
```

And then... nothing visible. No TUI. No status bar. The terminal just sits there. But tmux is running, and if you type a tmux command (like `list-windows`) and press Enter, you'd see:

```
%begin 1711000001 1 0
0: zsh* (1 panes) [80x24] [layout d962,80x24,0,0,0] @0 (active)
%end 1711000001 1 0
```

If someone types in a tmux pane, you'd see:

```
%output %0 hello\015\012
```

This is the **protocol**. Every message starts with `%`. Everything is plain text. A program can parse this and build its own UI.

1.6 The control mode protocol messages

1.6.1 Command blocks: `%begin` / `%end` / `%error`

Every command you send to tmux produces a response wrapped in guard lines.

Successful command:

```
%begin <timestamp> <command_number> <flags>
<output lines...>
%end <timestamp> <command_number> <flags>
```

Failed command:

```
%begin <timestamp> <command_number> <flags>
<error message>
%error <timestamp> <command_number> <flags>
```

- `<timestamp>` — seconds since Unix epoch (integer)
- `<command_number>` — sequential number, starting at 0
- `<flags>` — currently always 0 (reserved for future use)

The timestamp and command number on `%begin` must match those on the corresponding `%end` or `%error`. This lets a client correlate responses to commands.

Important subtlety (issue): A line inside a block payload might coincidentally start with `%end` or `%error` (e.g., a shell command that outputs the literal text `%end`). A correct parser must verify the **full guard-line format** (timestamp, command number, flags) — not just the `%end` prefix. This was the subject of Ghostty issue #11395 and a parser bug fix.

1.6.2 `%output` and `%extended-output`

```
%output %<pane_id> <escaped_data>
%extended-output %<pane_id> <ms_behind> : <escaped_data>
```

These carry terminal output from a pane. When a program running in a tmux pane produces output (text, escape sequences, etc.), tmux sends it to the control mode client as `%output`.

`%extended-output` is the flow-control-aware variant (see 1.6.5 below). It includes `<ms_behind>`, the number of milliseconds the pane's output is behind real-time.

The `<escaped_data>` contains the raw terminal output, but with escaping applied (see 1.6.4).

1.6.3 Async notifications

These are sent by tmux without being requested — they notify the client of state changes:

Session notifications:

Message	Meaning
<code>%session-changed \$<id> <name></code>	Client's attached session changed
<code>%session-renamed \$<id> <name></code>	A session was renamed
<code>%sessions-changed</code>	A session was created or destroyed
<code>%session-window-changed \$<id> @<wid></code>	A session's current window changed

Window notifications:

Message	Meaning
<code>%window-add @<id></code>	Window added to attached session
<code>%window-close @<id></code>	Window closed
<code>%window-renamed @<id> <name></code>	Window renamed
<code>%window-pane-changed @<wid> %<pid></code>	Active pane in a window changed

Message	Meaning
<code>%layout-change @<wid> <layout> <vis_layout> <flags></code>	Window layout changed (resize, split, etc.)

Unlinked window notifications (windows in other sessions):

Message	Meaning
<code>%unlinked-window-add @<id></code>	Window added to another session
<code>%unlinked-window-close @<id></code>	Window closed in another session
<code>%unlinked-window-renamed @<id> <name></code>	Window renamed in another session

Client notifications:

Message	Meaning
<code>%client-session-changed <client> \${<sid>} <name></code>	Another client changed session
<code>%client-detached <client></code>	Another client detached

Paste buffer notifications:

Message	Meaning
<code>%paste-buffer-changed <name></code>	A paste buffer changed
<code>%paste-buffer-deleted <name></code>	A paste buffer deleted

Subscription notifications:

Message	Meaning
<code>%subscription-changed <name> \${<sid>} @<wid> <idx> %<pid> : <value></code>	A subscribed format value changed

Flow control:

Message	Meaning
<code>%pause %<pane_id></code>	Pane paused (output suspended)
<code>%continue %<pane_id></code>	Pane resumed

Other:

Message	Meaning
<code>%pane-mode-changed %<pane_id></code>	Pane entered/exited copy mode
<code>%exit [reason]</code>	Client exiting

1.6.4 Escaping rules

In `%output` and `%extended-output` data **(doc)**:

- Control characters (ASCII < 32) are replaced with **octal escapes**: `\ooo` (three octal digits). For example, newline (0x0A) → `\012`, carriage return (0x0D) → `\015`.
- The backslash character `\` itself → `\134`.
- All other printable characters are passed through literally.

The pane output data may contain terminal escape sequences for the `TERM=tmux` or `TERM=screen` terminal type. The control mode client must decode the octal escaping first, then feed the resulting bytes into a terminal emulator (VT parser) to interpret them.

1.6.5 Flow control

Flow control prevents a slow client from being disconnected when panes produce output faster than the client can consume it **(doc)**.

Enabling: Send `refresh-client -f pause-after=<seconds>`

How it works:

1. When enabled, `%output` is replaced by `%extended-output` (which includes a "milliseconds behind" field).
2. When a pane's output falls behind by the specified duration, tmux sends `%pause %<pane_id>` and stops sending output for that pane.
3. The client resumes the pane with: `refresh-client -A '%<pane_id>:continue'`
4. On resume, tmux sends `%continue %<pane_id>`.

Manual pane control:

- `refresh-client -A '%<pid>:pause'` — manually pause
- `refresh-client -A '%<pid>:off'` — stop output entirely
- `refresh-client -A '%<pid>:on'` — re-enable

Internal limits (from tmux source):

- Without pause mode, a client that falls more than 300 seconds behind gets disconnected with `%exit` and reason "too far behind".

1.6.6 ID system

tmux uses stable numeric IDs with sigils **(doc)**:

- Sessions: `$<id>` (e.g., `$0`)
- Windows: `@<id>` (e.g., `@1`)
- Panes: `%<id>` (e.g., `%0`)

These are globally unique and stable (unlike names or indices).

1.6.7 DCS framing

When using `-CC`, the entire control mode session is wrapped in a DCS (Device Control String) escape sequence **(doc, code)**:

- **Entry:** tmux sends `ESC P 1000 p` (DCS with parameter 1000, final byte `p`). A terminal emulator can detect this to know control mode has started.
- **Exit:** tmux sends `%exit [reason]` followed by `ESC \` (ST, String Terminator).

This DCS framing is what allows a terminal emulator to distinguish control mode traffic from normal terminal output.

1.7 Comparison: three ways to use tmux

1. Normal tmux usage in a terminal:

```
You type → Terminal → tmux client → tmux server → shell
Shell output → tmux server → tmux client → (draws TUI) → Terminal → screen
```

The terminal is a dumb display. tmux draws everything.

2. tmux control mode as protocol:

```
You type → Control client → (sends commands) → tmux server → shell
Shell output → tmux server → (sends %output) → Control client → ???
```

The control client receives structured data. It must decide what to do with it. There is no TUI.

3. What Ghostty could provide on top of control mode:

```
You type → Ghostty → (sends keys to tmux) → tmux server → shell
Shell output → tmux server → %output → Ghostty parser → Ghostty terminal
emulator
tmux panes → native Ghostty tabs/splits
tmux windows → native Ghostty windows
Scrollback → native Ghostty scrollback
Selection → native Ghostty selection/clipboard
Search → native Ghostty search
```

Ghostty would replace the tmux TUI with native OS-level UI elements. The user would never see tmux's TUI — they'd get the persistence and remote benefits of tmux with the native look, feel, and integration of Ghostty.

1.8 Product perspective: what the user experience would be

If this were fully implemented:

1. User opens Ghostty and runs `tmux -CC attach` (or Ghostty starts it automatically).
2. Each tmux window appears as a native Ghostty tab.
3. Each tmux pane appears as a native Ghostty split.
4. The user can scroll with the native Ghostty scrollbar.
5. The user can search pane content with Ghostty's search.
6. The user can select and copy with native selection.
7. Closing a Ghostty tab closes the tmux window.

8. Opening a new tab creates a new tmux window.
9. Splitting a pane creates a new tmux pane.
10. If the user closes Ghostty and reopens, they can reattach and everything is back — because tmux kept the session alive.

This is exactly what iTerm2 provides today with `tmux -CC`.

Section 2: How People Use tmux Control Mode Today

2.1 Which terminals/tools support it?

iTerm2 (macOS): The most mature implementation. George Nachman (iTerm2's author) co-designed the control mode protocol with the tmux developers. This is the reference implementation. **(doc)**

WezTerm: Has a working implementation merged in PR #6602. Treats tmux as a "multiplexer domain" and renders native windows/tabs/panes. Still has some known issues (SSH hangs, WSL issues). **(issue)**

No other major terminals currently support it. Not Alacritty, not kitty, not Windows Terminal (which has open feature requests but no implementation). **(inferred from web search)**

2.2 Common use cases

1. **SSH workflow with native UI**: You SSH to a remote server. You run `tmux -CC attach`. Your remote tmux sessions appear as native tabs and splits in iTerm2. You get native scrollback, search, and copy-paste — all for remote sessions.
2. **Session persistence with native feel**: You close your laptop. Tomorrow, you open iTerm2 and run `tmux -CC attach`. Everything is back — same tabs, same panes, same content — but it all looks and feels like native iTerm2.
3. **Multi-window development**: You have a complex dev setup — editor pane, build pane, log pane, REPL pane. With control mode, each is a native split in your terminal emulator. You can resize with your mouse naturally, use native keyboard shortcuts, etc.

2.3 Step-by-step: how a user uses this today in iTerm2

1. Open iTerm2.
2. Type: `tmux -CC new-session -s work`
3. iTerm2 detects the DCS sequence and activates control mode.
4. A notification bar appears: "tmux integration active."
5. The current tmux window appears as a native iTerm2 tab.
6. Press Cmd+T — iTerm2 creates a new tmux window (sends `new-window` command to tmux).
7. Press Cmd+D — iTerm2 creates a new tmux pane (sends `split-window` command to tmux).
8. You work normally. Everything looks native.
9. Close iTerm2.
10. Open iTerm2 again. Type: `tmux -CC attach -t work`
11. All your tabs and panes reappear, with content intact.

2.4 Why choose control mode over normal tmux?

- **Native UI**: No box-drawing pane borders, no tmux status bar. Real OS-native tabs, splits, scrollback.

- **Native keybindings:** Cmd+C to copy, Cmd+V to paste, Cmd+F to search. No need to learn tmux's key bindings.
 - **OS integration:** Proper font rendering, native resize handles, accessibility support, system clipboard integration.
 - **Scrollback:** Native terminal scrollback instead of tmux's copy mode.
 - **Still get persistence:** All the benefits of tmux (session survival, remote reattach) without the TUI tradeoffs.
-

Section 3: User Story / Product Perspective

3.1 Who is this feature for?

1. **Remote developers** who SSH into servers and want session persistence with a native feel. **(issue #1935 — explicitly requested)**
2. **iTerm2 refugees** who use tmux -CC today and want the same experience in Ghostty. **(issue #1935 — dtenenba confirmed this)**
3. **Power users** who want tmux's session management but dislike its TUI aesthetics and keybinding conflicts.
4. **AI/agent workflows** that use tmux for multi-pane execution and want native terminal integration. **(issue #1935 — findepi mentioned Claude agent teams)**

3.2 What problem does it solve?

Today, using tmux in Ghostty means:

- You lose native Ghostty features inside tmux (scrollback, search, selection behavior).
- tmux draws its own UI — borders, status bar — which looks different from Ghostty's native UI.
- Keyboard shortcuts conflict (Ghostty keybindings vs tmux prefix keys).
- Scrolling behavior is different inside tmux.
- No OS-level integration for tmux's windows and panes.

Control mode integration would eliminate these frictions entirely.

3.3 What would a successful workflow look like?

(inferred from iTerm2 behavior and issue #1935 discussion)

1. User configures Ghostty to auto-start with `tmux -CC attach` (or triggers it manually).
2. Ghostty detects the DCS `ESC P 1000p` sequence.
3. Ghostty reads tmux's initial session state (windows, panes, layouts).
4. Each tmux window appears as a Ghostty tab.
5. Each tmux pane appears as a Ghostty split.
6. Content in each pane is populated from tmux (scrollback + visible area).
7. Ongoing output (`%output`) is routed to the correct pane's terminal.
8. User actions (new tab, close tab, resize, split) are sent as tmux commands.
9. Tmux notifications update Ghostty's UI accordingly.
10. Detaching or closing Ghostty leaves the tmux session running.
11. Reattaching restores everything.

3.4 What "native integration" means

- **Native tabs** = tmux windows → Ghostty tabs (macOS native tab bar)
- **Native splits** = tmux panes → Ghostty split views
- **Native scrollbar** = Ghostty manages scrollbar per pane, populated from tmux history
- **Native search** = Cmd+F searches the Ghostty-managed buffer
- **Native selection** = click-and-drag, Cmd+C, Cmd+V
- **Native resize** = drag split handles, resizes both Ghostty view and tmux pane

3.5 Evidence vs inference

Claim	Basis
Users want this feature	(issue) #1935, 50+ participants
iTerm2 provides this today	(doc) iTerm2 documentation
WezTerm is implementing it	(issue) wezterm PR #6602
It would eliminate tmux TUI friction	(inferred) from how control mode works
Ghostty tabs would map to tmux windows	(inferred) from iTerm2's approach, but Ghostty's exact UI is not documented
It would improve remote SSH workflows	(issue) explicitly mentioned in #1935

Section 4: Protocol Checklist

This checklist covers what a serious control mode client needs to handle.

4.1 Connection lifecycle

Element	Meaning	Priority	Client must...
DCS <code>ESC P 1000p</code> detection	Entry to control mode	Essential	Detect DCS, switch to control mode parsing
<code>%exit [reason]</code>	Client exiting	Essential	Clean up, optionally show reason to user
ST (<code>ESC \</code>) after exit	DCS termination	Essential	Exit control mode parsing
<code>wait-exit</code> flag	tmux waits for empty line before exiting	Useful	Optionally send empty line after cleanup
Sending empty line to detach	Voluntary detach	Essential	Support user-initiated detach

4.2 Command/response blocks

Element	Meaning	Priority	Client must...
<code>%begin <ts> <cmd> <flags></code>	Start of command response	Essential	Track block start, record command number

Element	Meaning	Priority	Client must...
<code>%end <ts> <cmd> <flags></code>	Successful command end	Essential	Correlate with begin, process block content
<code>%error <ts> <cmd> <flags></code>	Failed command end	Essential	Correlate with begin, handle error
Block payload parsing	Content between begin/end	Essential	Parse command-specific output
Guard-line validation	Verify full <code>%end/%error</code> format	Essential	Avoid false termination on payload lines starting with <code>%end</code>

4.3 Pane output

Element	Meaning	Priority	Client must...
<code>%output %<pid> <data></code>	Pane terminal output	Essential	Decode octal escaping, feed to VT parser for that pane
<code>%extended-output</code>	Flow-controlled pane output	Useful	Same as <code>%output</code> , also track latency
Octal escape decoding	<code>\ooo</code> for control chars, <code>\134</code> for <code>\</code>	Essential	Correctly decode all escaped bytes
<code>%pane-mode-changed %<pid></code>	Pane entered/exited copy mode	Situational	Optionally adjust UI

4.4 Session notifications

Element	Meaning	Priority	Client must...
<code>%session-changed \$<id> <name></code>	Attached session changed	Essential	Update session state, refresh all windows
<code>%session-renamed</code>	Session renamed	Useful	Update displayed session name
<code>%sessions-changed</code>	Session created/destroyed	Useful	Update session list if exposed in UI

4.5 Window notifications

Element	Meaning	Priority	Client must...
<code>%window-add @<id></code>	Window added	Essential	Create new tab/window
<code>%window-close @<id></code>	Window closed	Essential	Close tab/window
<code>%window-renamed @<id> <name></code>	Window renamed	Useful	Update tab/window title
<code>%layout-change @<wid> ...</code>	Layout changed	Essential	Reparsing layout, resize/add/remove panes

Element	Meaning	Priority	Client must...
<code>%window-pane-changed @<wid> %<pid></code>	Active pane changed	Useful	Update focus
<code>%session-window-changed \$<sid> @<wid></code>	Current window changed	Useful	Switch active tab

4.6 Client notifications

Element	Meaning	Priority	Client must...
<code>%client-session-changed</code>	Another client changed session	Situational	Informational only
<code>%client-detached</code>	Another client detached	Situational	Informational only

4.7 Paste buffer notifications

Element	Meaning	Priority	Client must...
<code>%paste-buffer-changed</code>	Paste buffer changed	Optional	Optionally sync with system clipboard
<code>%paste-buffer-deleted</code>	Paste buffer deleted	Optional	Optionally update clipboard state

4.8 Flow control

Element	Meaning	Priority	Client must...
<code>refresh-client -f pause-after=N</code>	Enable flow control	Useful	Send at startup for robustness
<code>%pause %<pid></code>	Pane paused	Useful	Track paused state
<code>%continue %<pid></code>	Pane resumed	Useful	Track resumed state
<code>refresh-client -A '%<pid>:continue'</code>	Resume pane	Useful	Send when ready to receive more

4.9 Subscriptions

Element	Meaning	Priority	Client must...
<code>refresh-client -B '...'</code>	Subscribe to format changes	Optional	Use for monitoring without polling
<code>%subscription-changed</code>	Subscribed value changed	Optional	Process if using subscriptions

4.10 Commands the client sends

Command	Purpose	Priority
<code>list-windows -F '...'</code>	Get all windows and layouts	Essential
<code>list-panes -F '...'</code>	Get pane state (cursor, modes)	Essential

Command	Purpose	Priority
<code>capture-pane -p -e -q ...</code>	Get pane content (scrollback + visible)	Essential
<code>display-message -p '...'</code>	Get server info (version, etc.)	Useful
<code>new-window</code>	Create window	Essential (for full integration)
<code>kill-window -t @<id></code>	Close window	Essential
<code>split-window</code>	Create pane	Essential
<code>kill-pane -t %<id></code>	Close pane	Essential
<code>resize-pane -t %<id> -x W -y H</code>	Resize pane	Essential
<code>send-keys -t %<id> ...</code>	Send input to pane	Essential
<code>refresh-client -C WxH</code>	Set client size	Essential
<code>select-window -t @<id></code>	Switch active window	Useful
<code>select-pane -t %<id></code>	Switch active pane	Useful

4.11 Layout string parsing

Element	Meaning	Priority	Client must...
Layout format (e.g. <code>80x24,0,0,42</code>)	Pane tree description	Essential	Parse to determine pane arrangement
Checksum (4-char hex prefix)	Layout integrity check	Useful	Validate checksum matches
Nested <code>{}/[]</code>	Horizontal/vertical splits	Essential	Parse recursive tree structure

Section 5: Ghostty's Current tmux Story

5.1 Normal tmux usage in Ghostty (as a regular terminal)

Ghostty is a fully capable terminal emulator. You can run tmux inside Ghostty the same way you'd run it in any terminal:

```
tmux new-session -s work
```

This works today. tmux takes over the Ghostty window and draws its TUI. You get:

- tmux's pane borders, status bar, key bindings
- tmux's internal scrollback (via copy mode)
- tmux's session persistence

Known issue (issue #10227, closed): Ghostty's scrollbar and search could previously access tmux's off-screen history (the alternate screen buffer), which should be invisible since tmux is an alternate-screen application. This was partially fixed in PR #10229.

5.2 tmux control mode support: what exists in the code

Ghostty has **substantial but incomplete** tmux control mode support. Here is what the code contains, verified from source (**code**):

5.2.1 Build-level feature flag

tmux control mode is gated behind a build option `tmux_control_mode`, which is `true` when the Oniguruma regex library is available (which it is in standard builds). **Source:** `src/terminal/build_options.zig:46`

5.2.2 DCS detection (working)

The DCS handler in `src/terminal/dcs.zig` detects `ESC P 1000 p` and routes bytes to the tmux control parser. **Source:** `src/terminal/dcs.zig:52-58`

5.2.3 Control mode protocol parser (working, well-tested)

`src/terminal/tmux/control.zig` — 839 lines. A state machine that parses tmux control mode output byte-by-byte. It handles:

- Block begin/end/error with guard-line validation
- All major notification types: `%output`, `%session-changed`, `%sessions-changed`, `%layout-change`, `%window-add`, `%window-renamed`, `%window-pane-changed`, `%client-detached`, `%client-session-changed`
- State transitions: idle → notification → block → idle
- Memory limits and broken-state handling

26 tests covering notification parsing, block handling, edge cases.

Known limitation (code, issue #11395): The parser had a bug where payload lines starting with `%end` or `%error` could prematurely terminate a block. This was fixed (the fix required checking the full guard-line format). You contributed to this fix.

Missing from parser (inferred):

- `%window-close` — not seen in Notification union
- `%session-renamed` — not seen in Notification union
- `%session-window-changed` — not seen
- `%pane-mode-changed` — not seen
- `%paste-buffer-changed` / `%paste-buffer-deleted` — not seen
- `%subscription-changed` — not seen
- `%pause` / `%continue` — not seen
- `%extended-output` — not seen
- `%unlinked-window-*` — not seen

These omissions may be intentional (not all are needed for a first implementation) but represent protocol coverage gaps.

5.2.4 Layout parser (working, well-tested)

`src/terminal/tmux/layout.zig` — 638 lines. Parses tmux layout strings (e.g., `80x24,0,0{40x24,0,0,1,40x24,40,0,2}`) into a tree structure. Includes CRC16 checksum validation.

31 tests covering single panes, splits, nesting, error cases, checksums.

5.2.5 Command output parser (working, well-tested)

`src/terminal/tmux/output.zig` — 590 lines. Parses tmux format-string output from commands like `list-windows` and `list-panes` into typed structs. Supports 31 tmux variables covering cursor state, terminal modes, mouse modes, scroll regions, IDs, etc.

41 tests.

5.2.6 Viewer / reconciliation loop (working, no GUI connection)

`src/terminal/tmux/viewer.zig` — 2283 lines. A state machine that:

1. Handles the startup handshake (initial block → session-changed → command queue).
2. Queries tmux for windows, panes, and state.
3. Creates local `Terminal` instances for each pane.
4. Populates pane content using `capture-pane`.
5. Syncs terminal modes (cursor, mouse, insert, wrap, etc.).
6. Handles live `%output` by routing to the correct pane's terminal.
7. Handles `%layout-change` by re-syncing pane structure.
8. Handles `%window-add` by refreshing window list.

This is the core cross-platform business logic. (**issue, PR #9860**): Mitchell's comment: "This sucked. The control mode protocol is difficult."

Limitations noted in code (code):

- TODO: Startup robustness — session and block can happen out of order (viewer.zig:19)
- TODO: Ignore `%output` for panes not yet initialized (viewer.zig:21)
- TODO: Track active window pane for initial focus (viewer.zig:23)
- NOTE: Notification order assumptions based on tmux source as of Dec 2025 — fragile (viewer.zig:26-35)

5.2.7 Stream handler integration (partially working)

`src/termio/stream_handler.zig` handles the connection from DCS parsing to the Viewer:

- `.enter` → creates a Viewer instance (line 396-404)
- `.exit` → destroys the Viewer (line 407-413)
- Other notifications → passed to `viewer.next()`, actions processed
- `.command` actions → queued as write requests back to tmux (line 447-454)
- `.windows` action → `// TODO — NOT IMPLEMENTED` (line 456-458)

This `// TODO` is **the critical missing piece at the integration layer**. When the Viewer determines which windows/panes exist, it emits a `.windows` action — but the stream handler does nothing with it. This is where the GUI glue would need to create native Ghostty tabs and splits.

5.3 What works today

Capability	Status	Evidence
Normal tmux usage in Ghostty	Works	Standard terminal emulation
DCS <code>ESC P 1000p</code> detection	Works	code: dcs.zig
Control mode protocol parsing	Works (major notifications)	code: control.zig, 26 tests
Layout string parsing	Works	code: layout.zig, 31 tests
Command output parsing	Works	code: output.zig, 41 tests
Session state machine	Works (internally)	code: viewer.zig
Pane content capture	Works (internally)	code: viewer.zig
Send commands back to tmux	Works	code: stream_handler.zig
Create native tabs/splits from tmux state	NOT IMPLEMENTED	code: stream_handler.zig:456-458, <code>// TODO</code>
Apprt API for non-exec surfaces	NOT IMPLEMENTED	issue #1935 checklist

5.4 What does NOT work today

1. **No visible output from control mode.** If you run `tmux -CC` in Ghostty today, the Viewer runs internally and syncs state, but nothing appears on screen. The `.windows` action is silently dropped. **(code)**
2. **No native tabs/splits.** The fundamental missing piece: Ghostty cannot yet create surfaces (tabs, splits) that are backed by tmux panes rather than local shell processes. Mitchell identified this as "the ability to create new apprt things (windows/splits/tabs) that aren't attached to a normal exec-based termio. Which is pretty fundamentally hard." **(issue #1935, OO-kat quoting Mitchell)**
3. **No user input routing.** Even if windows were displayed, there's no mechanism to capture keystrokes from a tmux-backed surface and send them to tmux via `send-keys`. **(inferred)**
4. **No resize handling.** No code sends `refresh-client -C WxH` when Ghostty is resized. **(inferred from code inspection)**
5. **Several notification types not parsed.** See 5.2.3 above.
6. **No flow control.** No support for `%pause`, `%continue`, `%extended-output`, or `refresh-client -f pause-after=N`. **(code — not in Notification union)**

5.5 Summary of current state

The **protocol/parsing layer is substantial and well-tested**. The **reconciliation/state machine is implemented**. The **GUI integration layer is the major gap** — specifically:

- The "apprt" (application runtime) layer needs to support creating surfaces that aren't backed by a subprocess.
- The stream handler needs to process `.windows` actions and create real UI elements.
- Input routing, resize handling, and various user interactions need to be wired up.

The maintainer's characterization that support is "decent" is accurate for the protocol and state management layers. The characterization that "some missing work may not be parser work, but app/GUI glue work" is strongly confirmed by the code — **the GUI glue is the primary remaining challenge**.

5.6 Maintainer's checklist status (from issue #1935)

Item	Status	PR
Control Mode DCS Parser	Done	#1946 (merged 2024-07-12)
Termio non-subprocess support	Partial	#1948 (merged 2024-07-15)
Apprt API for non-subprocess surfaces	Not started	—
Termio hooks for tmux notifications	Partial	#9860 (merged 2025-12-10)
Parser for tmux command output	Done	#9803 (merged 2025-12-04)
Reconciliation loop	Done	#9860 (merged 2025-12-10)

The two unchecked items — **Apprt API changes** and **full Termio non-subprocess support** — are the architectural work that would connect the working internal state machine to visible UI.

What's Next

Sections 6–14 (in the second pass) will cover:

- Detailed code inspection with guided reading route
- Protocol coverage gap table
- First engineering problem identification
- Smallest meaningful work slice
- How to study the code yourself
- Diagrams
- Local experiments
- Testing strategy
- Maintainer summary

Ghostty Tmux Control Mode: Deep Analysis (Sections 6–14)

Continuation of overview_1_5.md. Read that first for foundations.

Evidence classification:

- **(doc)** = official documentation or man pages
- **(code)** = verified by reading Ghostty source code
- **(issue)** = from GitHub issues, PRs, or discussions
- **(inferred)** = reasonable conclusion — labeled explicitly

Section 6: Inspect Ghostty's Implementation

6.1 File-by-file inventory

File 1: `src/terminal/tmux/control.zig` (840 lines)

Purpose: Low-level byte-by-byte parser for the tmux control mode protocol. Turns raw bytes into structured `Notification` values.

Layer: Parser / Protocol

Key structs:

- `Parser` — State machine with states: `idle`, `notification`, `block`, `broken`. Accumulates bytes in a buffer and emits `Notification` values.
- `Notification` — Tagged union with 12 variants: `enter`, `exit`, `block_end`, `block_err`, `output`, `session_changed`, `sessions_changed`, `layout_change`, `window_add`, `window_renamed`, `window_pane_changed`, `client_detached`, `client_session_changed`.

What it handles:

- `%begin/%end/%error` blocks with full guard-line validation (timestamp, command number, flags must be numeric; exact token count required; no extra tokens allowed) (**code: lines 153-184**)
- `%output <id> <data>` with regex parsing (**code: lines 213-243**)
- `%session-changed <id> <name>` (**code: lines 244-274**)
- `%sessions-changed` (no arguments) (**code: lines 275-283**)
- `%layout-change @<id> <layout> <vis_layout> <flags>` (**code: lines 284-321**)
- `%window-add @<id>` (**code: lines 322-351**)
- `%window-renamed @<id> <name>` (**code: lines 352-382**)
- `%window-pane-changed @<wid> %<pid>` (**code: lines 383-417**)
- `%client-detached <client>` (**code: lines 418-443**)
- `%client-session-changed <client> $<sid> <name>` (**code: lines 444-475**)
- Unknown notifications: logged and gracefully skipped (**code: lines 476-485**)

What it does NOT handle (protocol elements not in the `Notification` union):

- `%window-close` — **missing** (essential for full integration)
- `%session-renamed` — missing (useful)
- `%session-window-changed` — missing (useful for tab switching)
- `%pane-mode-changed` — missing (situational)
- `%paste-buffer-changed / %paste-buffer-deleted` — missing (optional)
- `%subscription-changed` — missing (optional)
- `%pause / %continue` — missing (useful for flow control)
- `%extended-output` — missing (useful for flow control)
- `%unlinked-window-*` — missing (situational)
- `%exit` reason string — intentionally dropped (**code: lines 506-510**)

Fragile assumptions / TODOs:

- TODO: Validate that `%begin` and `%end/%error` blocks match by timestamp and command number (**code: line 206**)
- Uses Oniguruma regex for each notification parse — functional but could be optimized with simpler string parsing

- Parser enters **broken** state on unexpected non-% byte in idle state — emits synthetic **exit** notification (**code: lines 84-86**)

Tests: 26 tests covering block parsing, all notification types, edge cases (misleading %end in payload, token count validation, numeric metadata validation, carriage return handling).

File 2: **src/terminal/tmux/layout.zig** (639 lines)

Purpose: Parse tmux layout description strings into a tree structure representing pane arrangement.

Layer: Parser / Protocol (sub-parser for layout strings)

Key structs:

- **Layout** — Tree node with **width**, **height**, **x**, **y**, and **content** (either **.pane** leaf with ID, or **.horizontal/.vertical** with child array).
- **Checksum** — CRC16 enum for layout integrity validation using tmux's rotate-right algorithm.

What it handles:

- Single pane: **80x24,0,0,42**
- Horizontal splits: **80x24,0,0{40x24,0,0,1,40x24,40,0,2}**
- Vertical splits: **80x24,0,0[80x12,0,0,1,80x12,0,12,2]**
- Arbitrary nesting depth
- Checksum validation: 4-char hex prefix before layout string
- All syntax error cases

Tests: 31 tests covering all layout shapes, nesting, syntax errors, checksum validation including known tmux layout checksums.

File 3: **src/terminal/tmux/output.zig** (591 lines)

Purpose: Parse tmux command output (from commands like **list-windows**, **list-panes**, **display-message**) into typed Zig structs using tmux format variables.

Layer: Semantics (command output interpretation)

Key types:

- **Variable** — enum of 31 tmux format variables (cursor position, shape, color, blinking; alternate screen state; terminal modes; mouse modes; scroll region; tab stops; pane/window/session IDs; version; layout).
- **FormatStruct(vars)** — comptime function that generates a struct type matching the requested variables.
- **parseFormatStruct(T, str, delim)** — parse delimited output into the generated struct.
- **comptimeFormat(vars, delim)** — generate the tmux format string at compile time.

What it handles:

- Boolean flag parsing (**"1"**→true, anything else→false)
- Numeric parsing (usize)
- ID parsing with sigil stripping (**\$42**→42, **@3**→3, **%0**→0)
- String pass-through for layout, color, shape, version, tabs

File 4: `src/terminal/tmux/viewer.zig` (2284 lines)

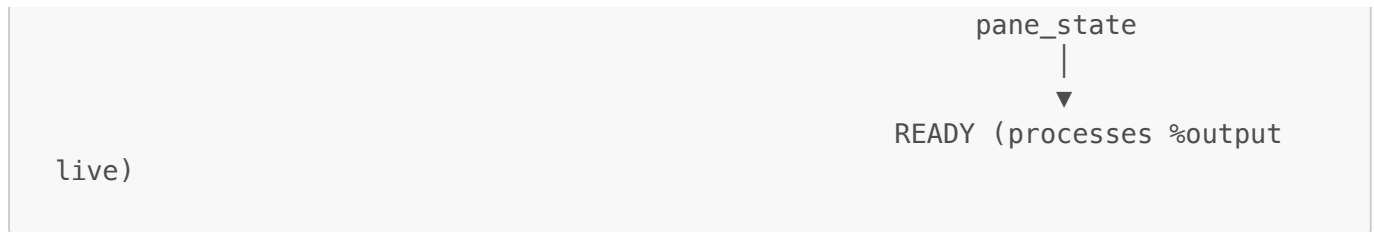
Layer: Semantics / App integration

- **Viewer** — The main struct. Contains:
 - **state**: **State** — **startup_block**, **startup_session**, **command_queue**, **defunct**
 - **session_id**: **usize** — current tmux session
 - **tmux_version**: **[]const u8**
 - **command_queue**: **CircBuf(Command)** — serialized command queue
 - **windows**: **ArrayList(Window)** — all windows in the session
 - **panes**: **AutoArrayHashMap(usize, Pane)** — all panes, keyed by ID
- **Window** — **id**, **width**, **height**, **layout**: **Layout**, **layout_arena**: **ArenaAllocator.State**
- **Pane** — contains a **Terminal** instance (full terminal emulator)
- **Action** — tagged union: **exit**, **command**: **[]const u8**, **windows**: **[]const Window**
- **Input** — tagged union: **tmux**: **control.Notification**
- **Command** — internal: **list_windows**, **pane_history**, **pane_visible**, **pane_state**, **tmux_version**, **user**

```

graph TD
    DCS[DCS 1000p detected] --> SB[startup_block]
    SB -- "%block_end" --> SS[startup_session]
    SS -- "%session-changed" --> Main["(main state)"]
    Main --> LW[list_windows]
    LW -- "%output" --> LW
    LW --> Sync[syncLayouts]
    Sync --> PH["pane_history (pri + alt)"]
    Sync --> PV["pane_visible (pri + alt)"]
    PH --> Join(( ))
    PV --> Join
    Join --> LChange["%layout-change"]
    LChange --> SB

```



What it handles:

- Startup handshake (block → session → command queue)
- Querying tmux version via `display-message`
- Listing windows via `list-windows -F '...'`
- Parsing window layouts and creating pane tree
- Creating local `Terminal` instances for each pane (correct dimensions)
- Capturing scrollbar history via `capture-pane -p -e -q -S - -E -1`
- Capturing visible area via `capture-pane -p -e -q`
- Both primary and alternate screen for each pane
- Syncing terminal state via `list-panes -F '...'` (cursor position, shape, blinking, visibility; insert/wrap/keypad/origin modes; all mouse modes; focus, bracketed paste; scroll region; tab stops)
- Routing live `%output` to correct pane's Terminal via VT stream
- Handling `%layout-change` by re-parsing layout, syncing panes, queuing captures for new panes, pruning removed panes
- Handling `%window-add` by re-querying `list-windows`
- Handling `%session-changed` by fully resetting the Viewer and starting over (preserving tmux version)
- Command queue: sends one command at a time, waits for `%begin/%end` response before sending next
- Error handling: any unrecoverable error transitions to `defunct` state and emits `.exit` action

What it does NOT handle:

- `%window-close` — not in the Notification union (**code**)
- `%session-window-changed` — not handled (no active tab tracking)
- Active pane tracking / initial focus (**code: TODO line 23**)
- Ignoring `%output` for panes not yet initialized (**code: TODO line 21**)
- Resize handling / `refresh-client -C` — no resize logic present
- Input routing (`send-keys`) — no mechanism to receive keystrokes
- Flow control (`refresh-client -f pause-after=N`) — not implemented
- User-initiated actions (new-window, split-window, kill-pane) — the `user` command variant exists but nothing drives it

Fragilities noted in code:

- Notification order based on tmux source as of Dec 2025, not formally documented (**code: lines 26-35**)
- Startup assumes block before session notification (**code: TODO line 19**)
- Max cols/width overflow uses unchecked `@intCast` (**code: line 1149-1151**)

Tests: 8 test cases using a `TestStep` + `testViewer` framework:

- `immediate exit` — exit during startup
- `session changed resets state` — full session switch
- `initial flow` — complete startup through pane capture with content verification
- `layout change` — split change adding pane
- `layout_change does not return command when queue not empty`

- `layout_change` returns `command` when queue was empty
- `window_add` queues `list_windows` when queue empty
- `window_add` queues `list_windows` when queue not empty
- `two pane flow with pane state` — full flow with terminal state sync

File 5: `src/terminal/dcs.zig` (431 lines)

Purpose: DCS (Device Control String) handler that detects tmux control mode entry (`ESC P 1000 p`) and routes bytes to the tmux control parser.

Layer: Parser integration

What it handles:

- Detects DCS with `params=[1000]`, `final='p'` → creates `ControlParser` and returns `.enter` notification (**code: lines 54-75**)
- Routes subsequent bytes to `ControlParser.put()` (**code: lines 130-134**)
- On DCS unhook (ST received): deinit parser, returns `.exit` (**code: lines 168-171**)

Tests: 5 DCS tests including tmux enter and implicit exit.

File 6: `src/termio/stream_handler.zig` (~461 relevant lines)

Purpose: Bridges the terminal stream (DCS/escape sequence parsing) to the tmux Viewer. This is where parsed DCS commands become Viewer actions.

Layer: App integration / Glue

What it handles:

- `.enter` → creates Viewer instance (**code: lines 396-404**)
- `.exit` → destroys Viewer (**code: lines 407-413**)
- Other notifications → feeds to `viewer.next()`, processes actions (**code: lines 437-460**)
- `.command` actions → queues write requests back to tmux process (**code: lines 447-454**)

THE CRITICAL GAP:

```
.windows => {
    // TODO
},
```

(code: lines 456-458)

This is where the Viewer says "here are the tmux windows/panes" and the stream handler... does nothing. This is the exact point where GUI glue is missing.

File 7: `src/terminal/tmux.zig` (14 lines)

Purpose: Module re-export file. Makes `ControlParser`, `ControlNotification`, `Layout`, `Viewer` available as `terminal.tmux.*`.

File 8: `src/terminal/main.zig` (line 26)

Purpose: Conditionally exports the tmux module:

```
pub const tmux = if (options.tmux_control_mode) @import("tmux.zig") else
struct {};
```

File 9: `src/terminal/build_options.zig` (line 46)

Purpose: Ties `tmux_control_mode` to Oniguruma availability:

```
opts.addOption(bool, "tmux_control_mode", self.oniguruma);
```

File 10: `src/termio/backend.zig`

Purpose: Defines the backend interface for terminal I/O. Currently only has `exec` (subprocess/pty).

Key code:

```
pub const Kind = enum { exec };
pub const Backend = union(Kind) { exec: termio.Exec };
```

Relevance: A tmux backend would need to be added here as a new `Kind` variant that drives surfaces from tmux control mode instead of a pty subprocess.

File 11: `src/apprt/` directory

Purpose: Application runtime — platform-specific UI framework integration (GTK, macOS AppKit via Swift bindings).

Relevance: The `apprt` layer creates `Surface` objects that are always backed by a `termio.Exec` backend today. Creating tmux-backed surfaces requires changes here.

Key points from code inspection:

- `Surface.init()` always creates a `termio.Exec` backend
 - `apprt.action.Action` has `new_window`, `new_tab`, `new_split` etc.
 - `apprt.surface.Message` carries messages between termio and UI
 - The boundary between termio and `apprt` is clean and message-based
-

6.2 Guided reading route

Read the files in this order. Each step builds on the previous one.

Step 1: `src/terminal/tmux/control.zig`

- **Why first:** This is the entry point of all tmux data. Everything starts as raw bytes and becomes `Notification` values here.
- **Question it answers:** "What does the tmux protocol look like from Ghostty's perspective, and how is it parsed?"
- **What to focus on:** The `Notification` union (what types exist), the `Parser` state machine (idle→notification→block), the `put()` method, and the `parseNotification()` function with its regex patterns.
- **Skim:** The `format()` function (debug printing), individual regex details (they're repetitive).

Step 2: `src/terminal/tmux/layout.zig`

- **Why second:** Layouts are critical to understanding how tmux describes its pane arrangement. The `Viewer` depends heavily on this.
- **Question it answers:** "How does tmux describe pane layouts, and how does Ghostty parse them into a tree?"
- **What to focus on:** The `Layout` struct and its `Content` union (pane vs horizontal/vertical), the `parse()` function, the checksum.
- **Skim:** Individual test cases (there are 31, they're repetitive).

Step 3: `src/terminal/tmux/output.zig`

- **Why third:** This handles command responses. After you understand notifications and layouts, you need to understand how Ghostty interprets command output.
- **Question it answers:** "How does Ghostty turn `list-windows` and `list-panes` output into typed data?"
- **What to focus on:** The `Variable` enum (what tmux state Ghostty cares about), the `FormatStruct` mechanism, `parseFormatStruct()`.
- **Skim:** Individual variable tests (very repetitive).

Step 4: `src/terminal/tmux/viewer.zig`

- **Why fourth:** This is the brain. Now that you understand notifications, layouts, and command output, you can follow the `Viewer`'s state machine.
- **Question it answers:** "How does Ghostty orchestrate a tmux control mode session from startup to steady state?"
- **What to focus on:**
 1. The ASCII lifecycle diagram (lines 55-144) — read this carefully
 2. The `State` enum and `next()` dispatch (lines 314-337)
 3. `nextStartupBlock` and `nextStartupSession` (lines 339-401)
 4. `nextCommand` (lines 416-557) — the main operating loop
 5. `receivedListWindows` → `syncLayouts` → `initLayout` — the window discovery flow
 6. `receivedPaneHistory`, `receivedPaneVisible` — how pane content is populated
 7. `receivedPaneState` — how terminal modes are synced
 8. `receivedOutput` — how live output is routed
 9. The `Action` union — what the `Viewer` tells its caller to do
 10. The tests (lines 1496-2283) — these are the best documentation of expected behavior

- **Skim on first pass:** `sessionChanged`, `layoutChanged` (complex but secondary).

Step 5: `src/terminal/dcs.zig`

- **Why fifth:** Now you understand the tmux stack. This file shows how it connects to the broader terminal parser.
- **Question it answers:** "How does Ghostty detect `tmux -CC` and start routing bytes to the control parser?"
- **What to focus on:** The `tryHook` function (line 50-110), the tmux detection logic (`params=[1000]`, `final='p'`), the `put` → `tryPut` chain.

Step 6: `src/termio/stream_handler.zig`

- **Why sixth:** This is where the rubber meets the road (or doesn't). This file shows the integration gap.
- **Question it answers:** "How is the Viewer connected to the rest of Ghostty, and where does it break down?"
- **What to focus on:** Lines 389-461, especially the `.windows => { // TODO }` at line 456.

Step 7: `src/termio/backend.zig`

- **Why seventh:** Understanding the backend architecture tells you what the GUI glue layer needs.
- **Question it answers:** "What would a tmux backend need to look like?"
- **What to focus on:** The `Kind` enum, the `Backend` union, the method signatures.

Section 7: Map Ghostty Support Against Protocol Checklist

Protocol Feature	Expected Client Behavior	Ghostty Status	Evidence	Missing Layer	Confidence
DCS <code>ESC P 1000p</code> detection	Detect, switch to control mode	Supported	<code>dcs.zig:54-75</code>	—	High
<code>%exit [reason]</code>	Clean up, emit exit	Supported (reason dropped)	<code>control.zig:504-511</code>	—	High
ST after exit	Exit DCS parsing	Supported	<code>dcs.zig:168-171</code>	—	High
<code>%begin/%end</code> blocks	Track, correlate, parse content	Supported	<code>control.zig:201-212, 108-139</code>	—	High
<code>%error</code> blocks	Handle failed commands	Supported	<code>control.zig:131-134</code>	—	High
Guard-line validation	Reject false <code>%end/%error</code> in payload	Supported	<code>control.zig:153-184</code>	—	High

Protocol Feature	Expected Client Behavior	Ghostty Status	Evidence	Missing Layer	Confidence
Begin/end matching by ID	Verify begin matches end tokens	Not validated	control.zig:206 TODO	Parser	High
%output %<pid> <data>	Decode, route to pane terminal	Supported (decode not verified)	control.zig:213-243, viewer.zig:1100-1115	—	Medium
%extended-output	Flow-controlled output	Unsupported	Not in Notification union	Parser	High
Octal escape decoding in %output	\ooo → byte	Unclear	Not visible in control.zig; may be handled elsewhere or may be missing	Parser?	Low
%session-changed	Update session, refresh all	Supported	control.zig:244-274, viewer.zig:720-758	—	High
%session-renamed	Update displayed name	Unsupported	Not in Notification union	Parser	High
%sessions-changed	Informational	Supported (ignored intentionally)	control.zig:275-283, viewer.zig:516	—	High
%session-window-changed	Switch active tab	Unsupported	Not in Notification union	Parser	High
%window-add @<id>	Create tab, refresh	Supported	control.zig:322-351, viewer.zig:613-621	—	High
%window-close @<id>	Close tab	Unsupported	Not in Notification union	Parser + Semantics	High
%window-renamed @<id> <name>	Update tab title	Parsed only (ignored in Viewer)	control.zig:352-382, viewer.zig:519	Semantics	High
%layout-change	Reparsing layout, sync panes	Supported	control.zig:284-321, viewer.zig:567-609	—	High
%window-pane-changed	Update focus	Parsed only (ignored in Viewer)	control.zig:383-417, viewer.zig:510	Semantics	High

Protocol Feature	Expected Client Behavior	Ghostty Status	Evidence	Missing Layer	Confidence
<code>%client-detached</code>	Informational	Supported (ignored intentionally)	control.zig:418-443, viewer.zig:523	—	High
<code>%client-session-changed</code>	Informational	Supported (ignored intentionally)	control.zig:444-475, viewer.zig:524	—	High
<code>%paste-buffer-changed</code>	Sync clipboard	Unsupported	Not in Notification union	Parser + Semantics	High
<code>%paste-buffer-deleted</code>	Update clipboard	Unsupported	Not in Notification union	Parser + Semantics	High
<code>%pane-mode-changed</code>	Adjust UI	Unsupported	Not in Notification union	Parser	High
<code>%pause %<pid></code>	Track paused state	Unsupported	Not in Notification union	Parser + Semantics	High
<code>%continue %<pid></code>	Track resumed state	Unsupported	Not in Notification union	Parser + Semantics	High
<code>%subscription-changed</code>	Process format changes	Unsupported	Not in Notification union	Parser + Semantics	High
<code>%unlinked-window-*</code>	Informational	Unsupported	Not in Notification union	Parser	High
<code>list-windows -F</code> output	Parse window info	Supported	viewer.zig:845-902	—	High
<code>list-panes -F</code> output	Parse pane state	Supported	viewer.zig:904-1033	—	High
<code>capture-pane -p -e -q</code>	Populate pane content	Supported	viewer.zig:1035-1098	—	High
<code>display-message -p</code>	Query version	Supported	viewer.zig:823-843	—	High
<code>send-keys -t %<id></code>	Route user input to pane	Unsupported	No input routing code	Semantics + UI glue	High
<code>refresh-client -C WxH</code>	Report client size	Unsupported	No resize handling	Semantics + UI glue	High
<code>new-window / split-window</code>	Create window/pane	Unsupported	No user action handling	UI glue	High

Protocol Feature	Expected Client Behavior	Ghostty Status	Evidence	Missing Layer	Confidence
<code>kill-window / kill-pane</code>	Close window/pane	Unsupported	No user action handling	UI glue	High
<code>resize-pane -t %<id></code>	Resize pane	Unsupported	No resize handling	UI glue	High
<code>select-window / select-pane</code>	Switch focus	Unsupported	No focus management	UI glue	High
Flow control (<code>refresh-client -f</code>)	Prevent disconnect	Unsupported	No flow control	Semantics	High
Native tabs from tmux windows	Display tmux windows as tabs	Unsupported	<code>stream_handler.zig:456-458 // TODO</code>	UI glue	High
Native splits from tmux panes	Display tmux panes as splits	Unsupported	Same TODO	UI glue	High
Non-exec surfaces	Surfaces without subprocess	Unsupported	<code>backend.zig: Kind = enum { exec }</code>	Architecture	High

Summary of coverage

Category	Supported	Unsupported	Ignored intentionally
Protocol parsing	10 notification types	10 notification types	—
Command output	4 commands	—	—
State management	Startup, windows, panes, layout, output routing	Resize, input, focus, flow control	4 notification types
GUI integration	0 features	All features	—

Section 8: Identify the True First Engineering Problem

8.1 The problem

The true first engineering problem is **not** in the parser or the Viewer. It is in the **apprt/termio integration layer**: creating Ghostty surfaces (tabs, splits) that are backed by tmux panes rather than by local subprocess PTYs.

Evidence:

1. The parser handles all notifications needed for basic operation. **(code: control.zig)**
2. The Viewer's reconciliation loop works correctly — it creates local Terminal instances, populates them, routes live output. **(code: viewer.zig tests pass)**
3. The Viewer emits `.windows` actions with the correct window/pane data. **(code: viewer.zig)**
4. The stream handler silently drops `.windows` actions with `// TODO`. **(code: stream_handler.zig:456-458)**
5. Mitchell explicitly identified this as the blocker: "the ability to create new apprt things (windows/splits/tabs) that aren't attached to a normal exec-based termio. Which is pretty fundamentally hard." **(issue: #1935, comment by 00-kat quoting Discord)**
6. The Backend union is `enum { exec }` — only one kind. **(code: backend.zig)**

8.2 Why this is the correct first problem

- The parser and Viewer are already done and tested. Improving them is polish work, not blocking work.
- Without surfaces appearing on screen, nothing is visible. The feature is invisible to users no matter how good the protocol handling is.
- The maintainer's own checklist has "Apprt API changes to note non-subprocess-based surfaces" as an unchecked item.

8.3 What NOT to do yet

- Do **not** add more notification types to the parser. The missing ones (`%window-close`, `%session-renamed`, etc.) are real gaps but they are not the bottleneck.
- Do **not** try to implement flow control, input routing, or resize handling until surfaces are visible.
- Do **not** try to implement the full user interaction loop (new tab, close tab, split, etc.) as a first step.

8.4 What assumptions remain uncertain

1. **How should tmux-backed surfaces relate to the existing Surface lifecycle?** The current `Surface.init()` always creates a `termio.Exec` backend. It's unclear whether a tmux surface should use a new backend kind (e.g., `termio.Tmux`) or should work differently (e.g., the Viewer feeds data to existing surfaces through a different mechanism). **(inferred — needs maintainer input)**
2. **Should each tmux pane get its own Surface, or should one surface render multiple panes?** iTerm2 uses one native split per pane. The Viewer already has one Terminal per pane, suggesting one Surface per pane is intended. But this is an architectural decision that needs confirmation. **(inferred)**
3. **Should the existing Viewer's Terminal instances be reused, or should apprt surfaces create their own?** The Viewer creates `Terminal` instances for content. Apprt surfaces also have a `Terminal`. Duplication or handoff needs design. **(inferred)**
4. **How should the parent surface (where `tmux -CC` runs) relate to the tmux-backed child surfaces?** The parent surface has the DCS/control parser. The child surfaces need to receive `%output` data and send keystrokes back. The communication model needs design. **(inferred)**

Section 9: Smallest Meaningful Vertical Slice

9.1 Recommended slice

Make the Viewer's `.windows` action produce visible logging or a notification, and verify the full pipeline from `tmux -CC` through to the Viewer's internal state being populated.

More specifically:

Step A: Replace the `// TODO` at `stream_handler.zig:456–458` with structured logging that reports:

- Number of windows
- For each window: ID, dimensions, pane count
- For each pane: ID, dimensions, whether content has been captured

This is a 10-20 line change.

Step B: Manually test by running `tmux -CC attach` in Ghostty and observing the log output. Verify that:

- The DCS is detected
- The Viewer starts up
- Windows and panes are discovered
- Content capture commands are sent and received
- Live `%output` is routed

Step C: Write a focused integration test that exercises the `stream_handler` → `Viewer` → `action` pipeline, verifying that `.windows` actions are emitted with correct data.

9.2 Why this slice

- It's **narrow** — a few lines of code change, mostly logging.
- It's **testable** — you can verify it locally with `tmux`.
- It's **useful** — it confirms the full pipeline works end-to-end before investing in the complex apprt changes.
- It's **safe** — it changes no behavior, just adds observability.
- It **builds understanding** — you'll see real `tmux` control mode data flowing through Ghostty's parser → `viewer` → `stream handler`.
- It's a **foundation** for the real work — once you can see the `.windows` actions, you can start designing how to create surfaces.

9.3 Why not larger work

- Implementing a new backend kind or modifying `apprt` is a significant architectural change. Attempting it before confirming the pipeline works is risky.
- The maintainer may have specific ideas about the `apprt` design that should inform the approach. This slice gives you evidence to have that conversation productively.

9.4 After this slice

The natural next slice would be to discuss with the maintainer how non-exec surfaces should work, then implement the simplest possible proof-of-concept: one `tmux` pane → one Ghostty surface, read-only (no input routing), no `resize`.

Section 10: How to Build Your Own Understanding

10.1 Files to read (in order)

This is a more practical version of the guided reading route in 6.2.

1. **src/terminal/tmux/control.zig** — Read lines 1-50 (intro + Parser struct + State enum), then lines 64-147 (the `put()` method). This teaches you the fundamental parsing loop. Then read the **Notification** union (lines 498-597) to see all possible events.
2. **src/terminal/tmux/viewer.zig** — Read the lifecycle diagram (lines 55-144). Then read `init()` (line 268), `next()` (line 314), and `nextStartupBlock` / `nextStartupSession` (lines 339-401). Then `nextCommand` (lines 416-557) — this is the main loop. Finally, read the tests starting at line 1496.
3. **src/termio/stream_handler.zig** — Read only lines 389-461 (the `.tmux` handling in `dcsCommand`). This is short and critical.
4. **src/termio/backend.zig** — Read the **Kind** enum and **Backend** union to understand the backend interface.
5. **src/termio/Exec.zig** — Skim to understand what a working backend looks like. Focus on `threadEnter()` (subprocess start), `queueWrite()` (data to pty), and the read thread.

10.2 tmux docs to read

1. **tmux wiki — Control Mode**: <https://github.com/tmux/tmux/wiki/Control-Mode> This is the single most important document. Read it fully.
2. **tmux(1) man page — CONTROL MODE section**: Run `man tmux` and search for "CONTROL MODE".
3. **tmux source — control-notify.c**: Shows exactly which notifications tmux sends and when.
4. **tmux source — control.c**: Shows the protocol implementation, escaping, flow control.

10.3 Adding logging / trace prints

Where to add logging:

1. **src/termio/stream_handler.zig:456** — In the `.windows` TODO. Add:

```
.windows => |windows| {
    log.info("tmux windows changed: {} windows", .{windows.len});
    for (windows) |w| {
        log.info(" window id={} {}x{}", .{w.id, w.width, w.height});
    }
},
```

2. **src/termio/stream_handler.zig:438** — The action logging line already exists:

```
log.info("tmux viewer action={f}", .{action});
```

This will show every action the Viewer emits.

3. **src/termio/stream_handler.zig:393** — The notification logging:

```
log.info("tmux control mode event cmd={f}", .{tmux});
```

This shows every notification received.

How to see the logs:

Ghostty logs to stderr. Run Ghostty from a terminal to see them:

```
# From another terminal
/path/to/ghostty 2>&1 | grep tmux
```

Or set the log level:

```
GHOSTTY_LOG=info /path/to/ghostty 2>&1 | grep -E "(tmux|terminal_tmux)"
```

10.4 Types of traces that would be most illuminating

1. **Protocol trace:** Capture every **Notification** the parser emits, with timestamps. This shows you the exact sequence tmux sends.
2. **Command trace:** Log every command the Viewer sends to tmux (the **.command** actions). This shows the request side.
3. **State transition trace:** Log every Viewer state change (startup_block → startup_session → command_queue).
4. **Window/pane trace:** Log every **.windows** action with full window and pane details.

All of these already have **log.info** calls in the code at the right points — you just need to enable the logging and filter for the **terminal_tmux** scope.

10.5 Using a debugger

When to use: If a specific state transition is confusing or if the Viewer enters **defunct** unexpectedly.

Where to set breakpoints:

- **viewer.zig:defunct()** (line 1215) — catches all error paths
- **viewer.zig:nextCommand()** (line 416) — the main dispatch
- **stream_handler.zig:437** — where Viewer actions are processed
- **control.zig:put()** (line 64) — if you need byte-level tracing

How to debug Zig:

```
# Build with debug info
zig build -Doptimize=Debug
# Run under lldb
lldb ./zig-out/bin/ghostty
(lldb) breakpoint set --file viewer.zig --line 1215
(lldb) run
```


10.6 Using unit tests as observation tools

The existing tests in `viewer.zig` are excellent for this. You can:

- 1. **Add your own `TestStep` sequences** to explore behavior you're curious about. The test framework lets you construct any sequence of tmux notifications and verify what the Viewer does.
- 2. **Add `check` callbacks** that print internal state:

```
.check = (struct {
    fn check(v: *Viewer, actions: []const Viewer.Action) anyerror!void {
        std.debug.print("panes: {}\n", .{v.panes.count()});
        for (actions) |a| std.debug.print("action: {}\n", .{a});
    }
}).check,
```

- 3. **Run specific tests:**

```
zig build test -Dtest-filter="initial flow" 2>&1
```

10.7 Building tiny reproduction drivers

Create a simple Zig program that feeds hardcoded tmux control mode output into a `Parser` → `Viewer` pipeline and prints the results. This is the fastest way to experiment without running real tmux:

```
// test_tmux.zig - standalone driver
const control = @import("src/terminal/tmux/control.zig");
// feed bytes, print notifications
```

10.8 Observable behaviors to confirm the analysis

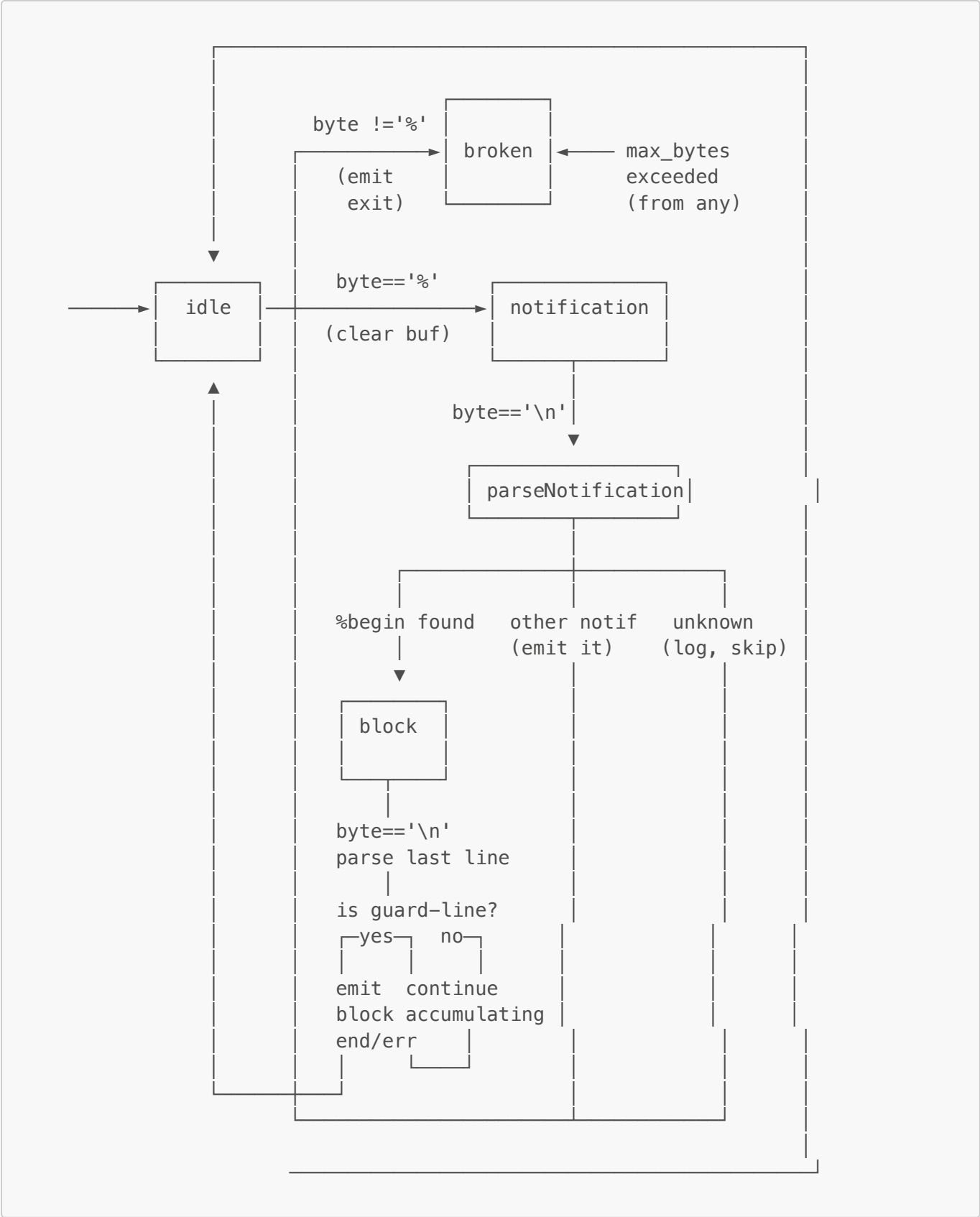
Behavior	What it confirms	How to test
Running <code>tmux -CC</code> in Ghostty shows no output but Ghostty logs show notifications	Parser and DCS detection work	Run <code>tmux -CC</code> , check logs
Logs show <code>%session-changed</code> , <code>list-windows</code> , <code>capture-pane</code>	Viewer startup works	Same as above
Logs show <code>.windows</code> action with correct window/pane IDs	Viewer reconciliation works	Same + add logging
Running <code>tmux split-window</code> in another client shows <code>%layout-change</code> in logs	Live notification handling works	Split from another tmux client
Viewer eventually reaches <code>command_queue</code> state with empty queue	Full initialization completes	Check logs or add state logging

Section 11: Recommended Diagrams and Thinking Tools

11.1 Parser state machine (most important)

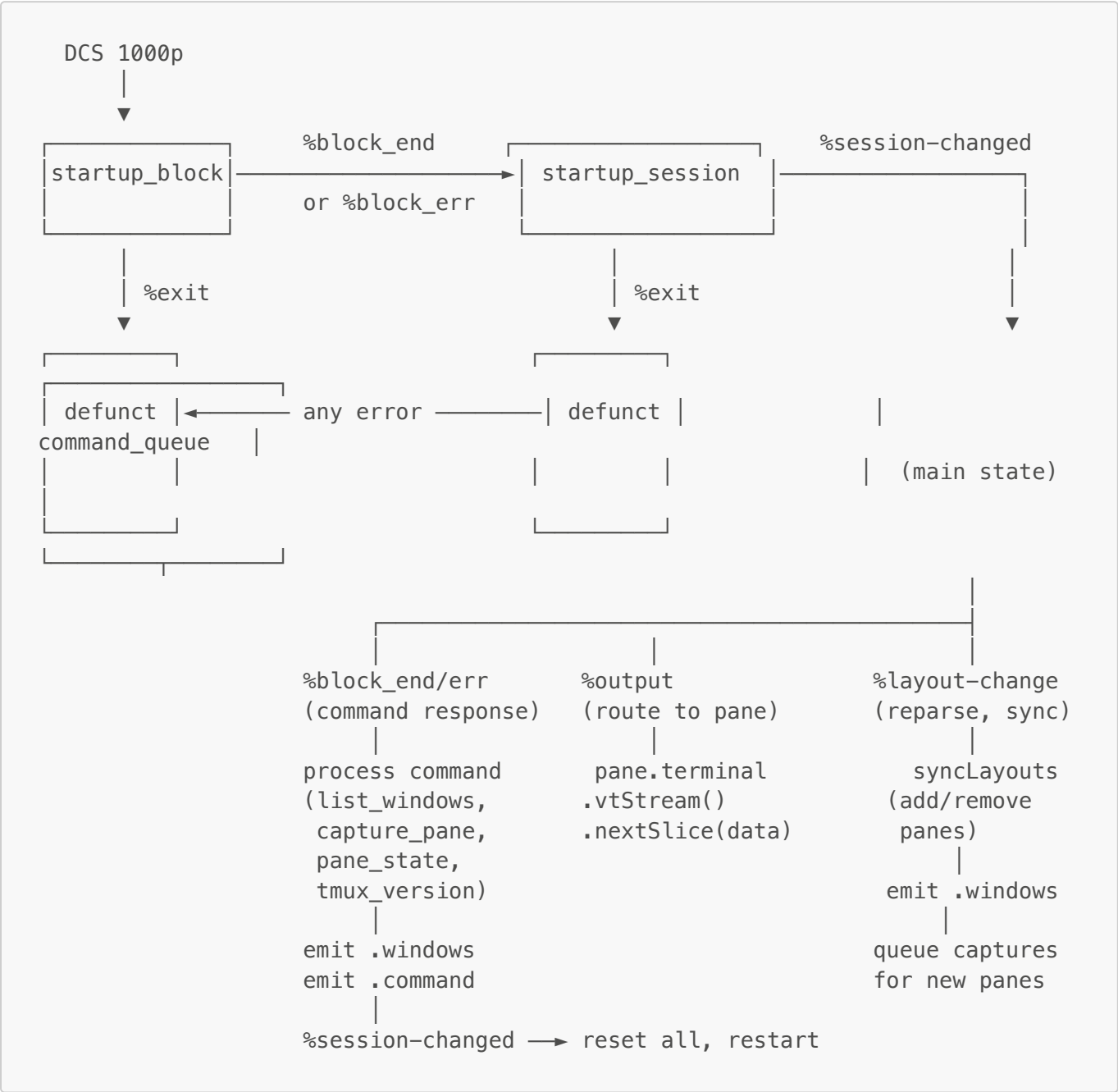
Why useful: The parser has only 4 states. Drawing them out makes the parsing logic completely clear and helps you verify edge cases.

What to draw: The 4 states and all transitions between them.



11.2 Viewer state machine

Why useful: Shows the entire lifecycle from connection to steady state. The Viewer's state machine is more complex than the parser's.



11.3 Data flow: DCS bytes → screen pixels (most important for understanding the gap)

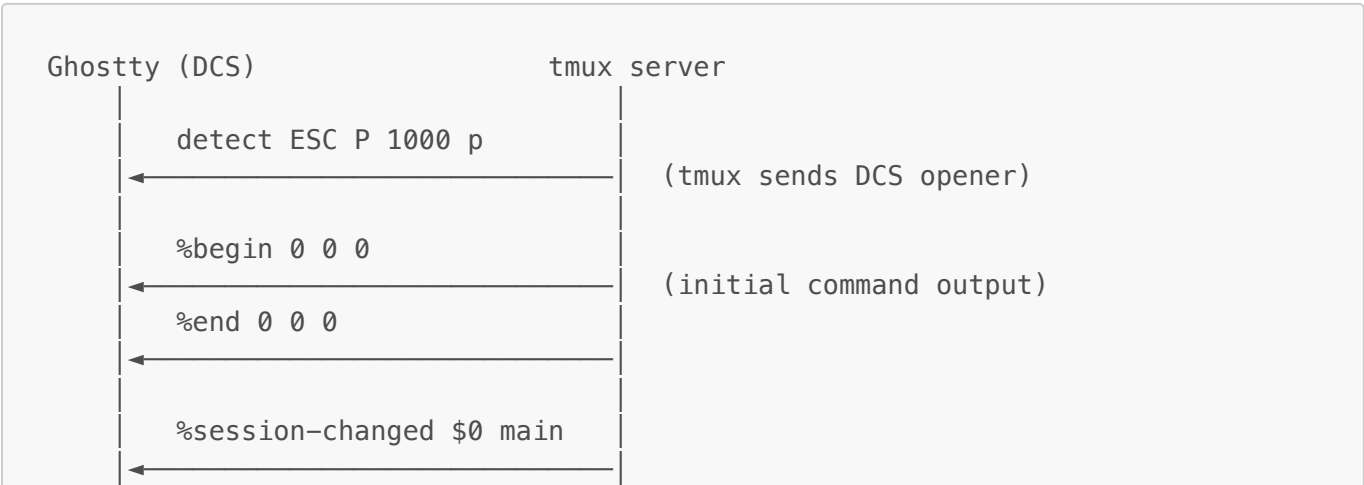
Why useful: Shows the full pipeline and exactly where the gap is.

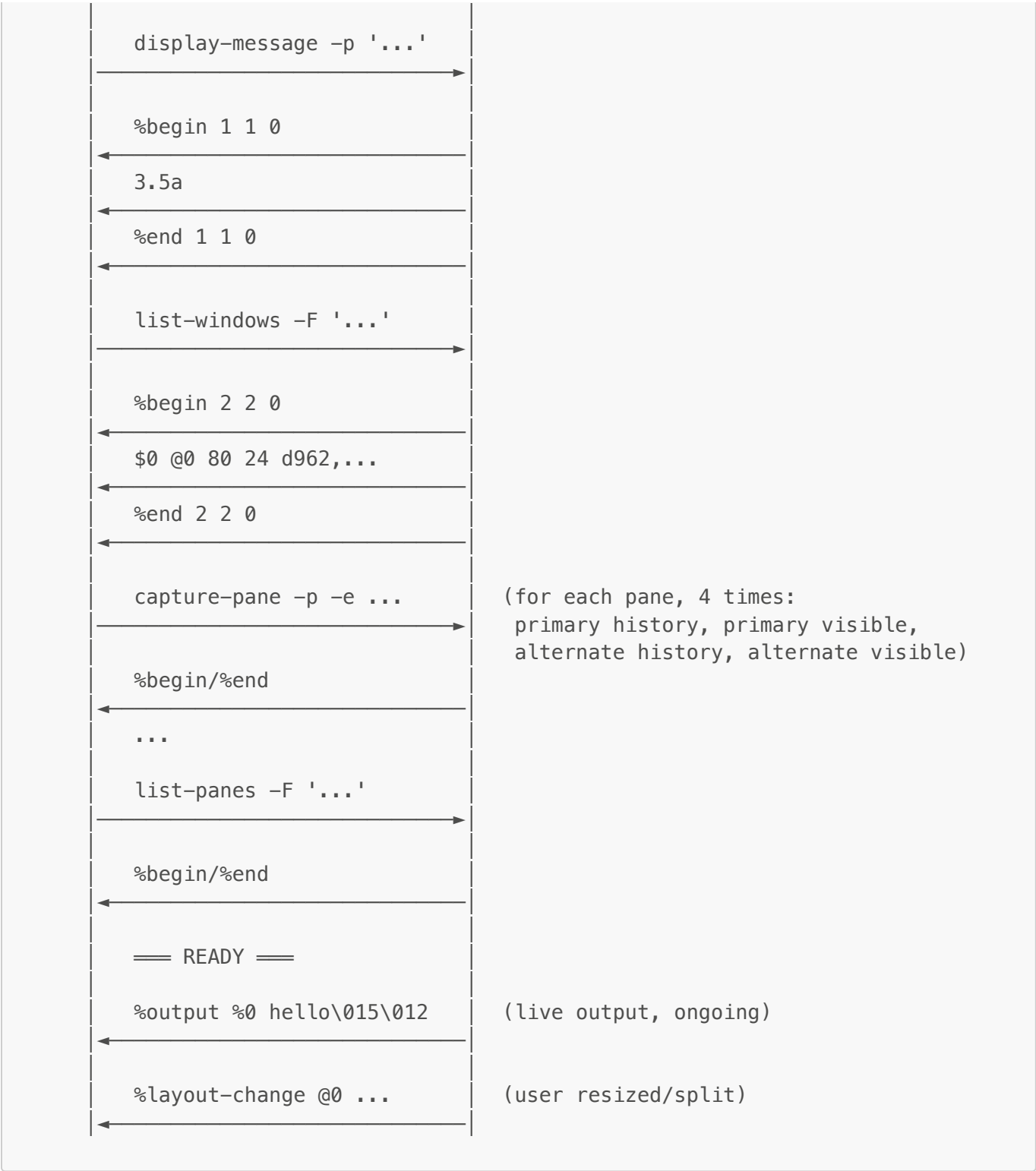




11.4 Command-response sequence diagram

Why useful: Shows the temporal ordering of the startup handshake, which is the most fragile part of the protocol.





11.5 Protocol trace timeline (for debugging)

Why useful: When debugging, log each event with timestamp and state to produce a timeline like:

```
T+0.000 DCS hook: params=[1000] final=p → state=tmux
T+0.001 Notification: enter
T+0.002 Viewer: startup_block
T+0.015 Notification: block_end("")
T+0.015 Viewer: startup_block → startup_session
T+0.020 Notification: session_changed($0, "main")
T+0.020 Viewer: startup_session → command_queue
T+0.020 Action: command("display-message -p '#{version}'\n")
```

```
T+0.025 Notification: block_end("3.5a")
T+0.025 Action: command("list-windows -F '...\n")
T+0.030 Notification: block_end("$0 @0 80 24 d962,80x24,0,0,0")
T+0.030 Action: windows([Window{id=0, 80x24, 1 pane}])
T+0.030 Action: command("capture-pane -p -e -q -S - -E -1 -t %0\n")
...
```

Section 12: How to Experiment Locally Today

Experiment 1: Normal tmux in Ghostty

What to run:

```
# In Ghostty
tmux new-session -s test
```

What you should expect to see:

- tmux takes over the Ghostty window
- Status bar at the bottom, shell prompt in the main area
- **Ctrl-b c** creates a new tmux window
- **Ctrl-b %** splits horizontally
- **Ctrl-b "** splits vertically

What conclusion to draw:

- Ghostty works as a normal terminal for tmux. No special integration — tmux just renders its TUI. This is the baseline experience.

Experiment 2: tmux control mode with echo (human-readable)

What to run:

```
# Start a regular tmux session first (in any terminal)
tmux new-session -d -s experiment

# In Ghostty, start control mode with echo
tmux -C attach -t experiment
```

What you should expect to see:

- You'll see protocol messages printed as text:

```
%begin 1711000000 0 0
%end 1711000000 0 0
%session-changed $0 experiment
```

- You can type commands like `list-windows` and see the block response
- If you open another terminal and run something in the session, you'll see `%output` lines

What conclusion to draw:

- This shows you the raw protocol. You can see exactly what tmux sends.
 - Ghostty treats this as normal terminal output — it doesn't intercept it because `-C` (single C) doesn't use DCS framing.
-

Experiment 3: tmux control mode with DCS (`-CC`)

What to run:

```
# Make sure you have a tmux session
tmux new-session -d -s experiment

# In Ghostty, start control mode with DCS
tmux -CC attach -t experiment
```

What you should expect to see:

- The terminal may appear to hang or show nothing visible
- If you run Ghostty from another terminal with log output enabled, you should see log lines about tmux control mode

What conclusion to draw:

- Ghostty detects the DCS `ESC P 1000p` and activates control mode
 - The Viewer starts up internally and processes notifications
 - But nothing appears on screen because `.windows` actions are dropped
 - This confirms the analysis: parser works, GUI glue is missing
-

Experiment 4: Observe the logs

What to run:

```
# Terminal A: start Ghostty with log output
/Applications/Ghostty.app/Contents/MacOS/ghostty 2>&1 | grep -i tmux

# Terminal B: in the Ghostty window that opened, run:
tmux -CC new-session -s logtest
```

What you should expect to see: In Terminal A, log lines like:

```
info(io_handler): tmux control mode event cmd=...
info(terminal_tmux_viewer): ...
info(io_handler): tmux viewer action=...
```

What conclusion to draw:

- The exact log output tells you which notifications are being parsed, which actions the Viewer emits, and whether the pipeline is working.
 - If you see `action=... .windows ...` in the logs, the Viewer is successfully discovering windows — it's just not displayed.
-

Experiment 5: Trigger notifications from another client**What to run:**

```
# Terminal A: Ghostty running tmux -CC attach -t logtest (with logs)

# Terminal B: connect to the same session normally
tmux attach -t logtest

# In Terminal B, do things:
echo "hello world"           # triggers %output
tmux split-window            # triggers %layout-change
tmux new-window              # triggers %window-add
tmux rename-window "test"    # triggers %window-renamed
```

What you should expect to see: In the logs from experiment 4, each action in Terminal B produces corresponding notification log lines in Ghostty's output.

What conclusion to draw:

- Each tmux operation produces specific protocol notifications
 - Ghostty's parser correctly identifies them
 - The Viewer processes them (or ignores them for unhandled types)
 - You can correlate tmux operations to protocol messages
-

Experiment 6: Inspect control mode protocol directly**What to run:**

```
# Use tmux -C (single C) to see raw protocol
tmux -C new-session -s raw

# Type these commands and observe output:
list-windows
list-panes
display-message -p '#{version}'
capture-pane -p -e -q -t %0
```

What you should expect to see: Each command produces `%begin/%end` blocks with output between them. For example:


```
%begin 1711000001 1 0
$0 @0 80 24 d962,80x24,0,0,0
%end 1711000001 1 0
```

What conclusion to draw:

- You can see the exact format strings and data that Ghostty's `output.zig` parsers need to handle.
 - Compare the format of real output to the format strings in `viewer.zig`'s `Format` struct (lines 1346-1419) to verify they match.
-

Experiment 7: Test the parser unit tests

What to run:

```
cd /Users/neurotone/code/ghostty

# Run all tmux-related tests
zig build test -Dtest-filter="tmux" 2>&1 | head -50
```

What you should expect to see: All tests pass. You'll see output indicating test execution.

What conclusion to draw:

- The parser, layout, output, and viewer unit tests all pass.
 - The test suite is comprehensive for the implemented functionality.
 - You can add new tests to explore edge cases.
-

Experiment 8: Examine the Viewer's internal Terminal content

What to run: Add a temporary test to `viewer.zig` that prints pane terminal content after the initial flow completes. Use the existing "initial flow" test as a template, adding print statements in the `check` callbacks.

Or modify an existing test to dump terminal content:

```
// In a check callback:
const pane = v.panes.getEntry(0)?.value_ptr;
const screen = pane.terminal.screens.active;
const str = try screen.dumpStringAlloc(testing.allocator, .{ .active = .{} });
defer testing.allocator.free(str);
std.debug.print("Pane 0 active content: '{s}'\n", .{str});
```

What you should expect to see: The pane's Terminal has the content from the `capture-pane` response, and live `%output` data appears after initial capture.

What conclusion to draw:

- The Viewer correctly populates Terminal instances with pane content.

- The internal state is ready to be rendered — it just needs a surface to render into.

Section 13: Testing and Validation Strategy

13.1 Parser unit tests (Layer A)

What exists: 26 tests in `control.zig` covering:

- Block begin/end with empty and non-empty payloads
- Guard-line validation (misleading payloads, token counts, numeric metadata)
- All 10 implemented notification types
- Carriage return handling

What's missing:

- Tests for unknown notification types (verify they're skipped gracefully) — partially covered by the catch-all else branch
- Tests for very large payloads approaching `max_bytes`
- Tests for the `broken` state recovery (or non-recovery)
- Tests for interleaved notifications and blocks (if tmux ever sends those)
- Tests for `%exit` with a reason string (currently dropped)

Recommended additions:

```
test "unknown notification is skipped" { ... }
test "max_bytes triggers broken state" { ... }
test "broken state drops all subsequent input" { ... }
```

13.2 Protocol conformance tests (Layer A+B)

Currently: Each notification is tested in isolation. There are no tests that feed a realistic tmux session transcript through the parser.

Recommended:

1. Capture a real tmux `-C` transcript and use it as a test input.
2. Feed the transcript byte-by-byte through the parser.
3. Verify the expected sequence of Notifications.

This would catch any issues with real-world protocol data that the isolated tests miss.

13.3 Viewer integration tests (Layer B+C)

What exists: 8 test cases in `viewer.zig` using the `TestStep` framework. They cover:

- Immediate exit
- Session change with state reset
- Initial flow (complete startup through content capture)
- Layout changes (adding panes)
- Command queue behavior (empty vs non-empty)
- Two-pane flow with terminal state sync

What's missing:

- Window close handling (requires `%window-close` parser support first)
- Multiple windows in one session
- Session switch mid-operation (while commands are in flight)
- Error recovery (tmux returns `%error` for a command)
- `%output` before pane initialization completes (race condition mentioned in TODO)
- Content verification after `%output` (the "initial flow" test checks `%output` routing but only verifies containment, not exact content)

Recommended additions:

```
test "output for uninitialized pane is ignored" { ... }
test "block_err for capture-pane is handled gracefully" { ... }
test "multiple windows in session" { ... }
```

13.4 Local experiments with tmux -C / -CC

See Section 12. These experiments serve as manual integration tests.

Key validation questions:

- Does `tmux -CC attach` trigger DCS detection? (Check logs)
- Does the Viewer reach `command_queue` state? (Check logs)
- Are windows and panes discovered correctly? (Compare logs to `tmux list-windows` output)
- Does live `%output` appear in logs? (Type in another client)

13.5 How to distinguish parser bugs from higher-level bugs

Symptom	Likely layer
Notification not parsed / "unknown notification" in logs	Parser — notification type not in <code>parseNotification()</code>
Notification parsed but Viewer ignores it	Semantics — <code>nextCommand()</code> has no handler for this type
Viewer processes correctly but nothing visible	UI glue — <code>.windows</code> action not handled
Viewer enters <code>defunct</code> unexpectedly	Semantics — error in command processing
Block content parsed incorrectly	Parser — guard-line validation or buffer handling issue
Wrong pane content	Semantics — VT stream routing or capture-pane handling

13.6 Regression test for issue #11395

The bug: Parser prematurely terminated blocks when payload contained lines starting with `%end` or `%error`.

The fix: Full guard-line validation (exact token count, numeric metadata).

Existing tests that cover this:

- "tmux block payload may start with %end" (control.zig:639)
- "tmux block payload may start with %error" (control.zig:654)
- "tmux block may terminate with real %error after misleading payload" (control.zig:669)
- "tmux block terminator requires exact token count" (control.zig:684)
- "tmux block terminator requires numeric metadata" (control.zig:699)

These tests are good. Additional edge cases to consider:

```
test "block payload line is exactly %end with no args" { ... }
test "block payload with %begin inside" { ... }
```

Section 14: Maintainer-Facing Summary

Here is a draft summary you could send to the maintainer.

Subject: tmux control mode — research findings and proposed next steps

Hi Mitchell,

I've done a deep analysis of Ghostty's tmux control mode support. Here's what I found:

What Ghostty already supports

The **protocol and state management layers are substantial and well-tested**:

- **DCS detection** (`dcs.zig`): Correctly detects `ESC P 1000 p` and routes to the control parser. Working and tested.
- **Control parser** (`control.zig`): Parses 10 notification types with full guard-line validation for `%begin/%end` blocks. 26 tests, including the `%end`-in-payload fix from issue #11395.
- **Layout parser** (`layout.zig`): Full recursive layout tree parsing with CRC16 checksum validation. 31 tests.
- **Command output parser** (`output.zig`): Handles 31 tmux format variables for `list-windows`, `list-panes`, `display-message`. 41 tests.
- **Viewer** (`viewer.zig`): Complete reconciliation state machine — startup handshake, window/pane discovery, content capture (primary + alternate screens), terminal mode sync (cursor, mouse, modes, scroll region, tab stops), live `%output` routing, layout change handling, session switching. 8 integration tests.
- **Stream handler integration** (`stream_handler.zig`): Creates/destroys Viewer, routes notifications, sends commands back to tmux.

Major gaps

1. **GUI glue** — `stream_handler.zig:456–458` has `// TODO` for the `.windows` action. The Viewer correctly discovers all windows/panes and emits actions, but nothing creates visible surfaces.
2. **Non-exec surfaces** — `backend.zig` only has `Kind = enum { exec }`. There's no way to create a surface that isn't backed by a subprocess/pty. This matches your note about needing "appert API changes to note non-subprocess-based surfaces."

3. **Input routing** — No `send-keys` support. Even if surfaces appeared, users couldn't type.
4. **Resize handling** — No `refresh-client -C WxH` or `resize-pane`.
5. **Missing notification types** — `%window-close`, `%session-renamed`, `%session-window-changed`, `%pause/%continue`, `%extended-output` are not in the parser's Notification union.

Which gaps are parser-related

- Missing notification types (#5 above) — straightforward additions to `control.zig`'s `parseNotification()` and `Notification` union.
- Begin/end block matching validation — noted as TODO in code.

Which gaps are app/GUI-related

- All of #1-4 above. These are the blockers.

Proposed next steps

1. **Immediate (small, safe):** Add structured logging to the `.windows` TODO to verify the full pipeline works end-to-end. Confirm that the Viewer correctly discovers windows/panes when running `tmux -CC` in Ghostty.
2. **Design discussion needed:** How should non-exec surfaces work? Options I see:
 - New `termio.Backend` kind (e.g., `.tmux`) where the Viewer's `Terminal` instances feed rendering
 - Viewer feeds data directly to existing surfaces through a shared Terminal pointer
 - Something else?
3. **First visible result:** Once the surface architecture is decided, the smallest proof-of-concept would be one `tmux` pane → one Ghostty surface (read-only, no input, no resize).

Open questions where your guidance would help

- What architecture do you envision for non-exec surfaces?
- Should the Viewer's `Terminal` instances be reused by surfaces, or should surfaces have their own?
- Should the parent surface (where `tmux -CC` runs) continue to exist as a visible surface, or should it be hidden?
- Are there apprt constraints (especially on macOS) that would affect the design?
- How do you want to handle the parent surface / child surface relationship for the `termio` write path (sending `send-keys` back)?

Happy to work on any of these — just want to make sure we align on architecture before investing in the complex parts.

End of maintainer summary draft.

Appendix: Quick Reference

File locations

File	Lines	Purpose
src/terminal/tmux/control.zig	840	Protocol parser
src/terminal/tmux/layout.zig	639	Layout tree parser
src/terminal/tmux/output.zig	591	Command output parser
src/terminal/tmux/viewer.zig	2284	Reconciliation state machine
src/terminal/tmux.zig	14	Module re-exports
src/terminal/dcs.zig	431	DCS handler / tmux detection
src/termio/stream_handler.zig	461 (relevant)	Integration glue
src/termio/backend.zig	~100	Backend interface
src/termio/Exec.zig	~large	Reference exec backend
src/apprt/action.zig	—	UI action definitions
src/apprt/surface.zig	—	Surface message contract

Key line numbers

Location	What's there
control.zig:498–597	Notification union (all event types)
control.zig:206	TODO: validate begin/end matching
viewer.zig:18–35	TODOs and fragility notes
viewer.zig:55–144	ASCII lifecycle diagram
viewer.zig:196–237	Action union (exit, command, windows)
viewer.zig:314–337	Main dispatch (<code>next()</code> → <code>nextTmux()</code>)
viewer.zig:416–557	<code>nextCommand()</code> — main operating loop
viewer.zig:1346–1419	Format strings for tmux commands
viewer.zig:1496–2283	All tests
stream_handler.zig:389–461	tmux DCS command handling
stream_handler.zig:456–458	THE TODO — .windows not implemented
backend.zig	<code>Kind = enum { exec }</code>

Test commands

```
# Run all tmux tests
zig build test -Dtest-filter="tmux"

# Run specific test
zig build test -Dtest-filter="initial flow"
```

```
# Run Ghostty with log output
/path/to/ghostty 2>&1 | grep -i tmux

# Start control mode for testing
tmux -C new-session -s test    # human-readable (no DCS)
tmux -CC new-session -s test   # DCS-framed (what Ghostty detects)
```